



Retrieval Augmented Generations

Kevin Bach, Sayantika Nag, Andy Chen
EE 269, UC Riverside CA



Outline of the presentation:

PART I:

- Introduction and Motivation
- Overview of RAG and its different architectures

PART II:

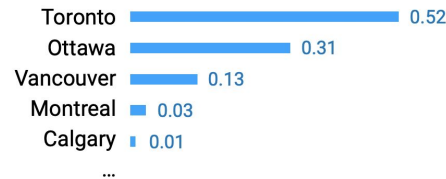
- In-depth analysis and results from some relevant papers



Large Language Models (LLMs)

- **Definition:** Deep learning algorithms trained on vast datasets such as trillions of words to recognize, summarize, translate, and generate content.
- **What it does:** They are essentially probabilistic next-token predictors. Based on the “context” you provide, they calculate the most likely word to follow, using the patterns they learned during training.

$$P(x_n | x_1, x_2, \dots, x_{n-1})$$



Language model (Transformers)

The capital city of Ontario is

x_1

x_2

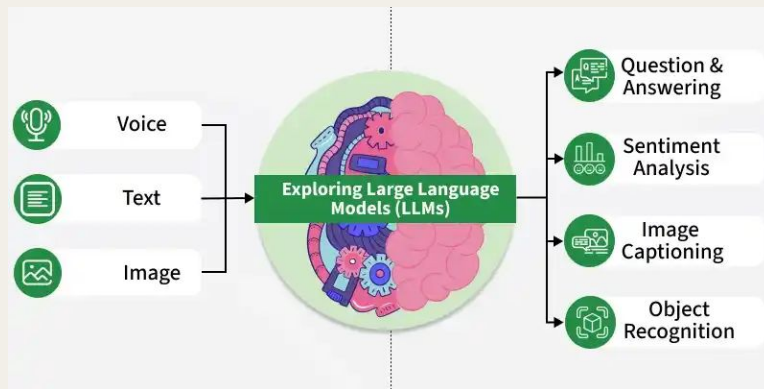
...

x_{n-1}



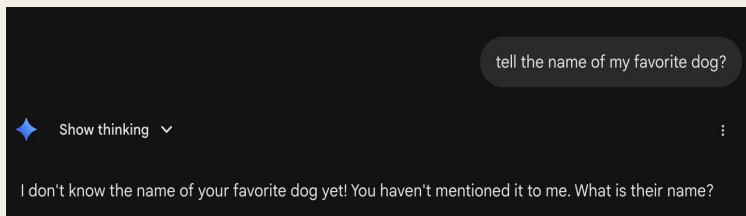
Why it's good and why we need it?

- **Natural Language Mastery:** LLMs understand the intent, nuance, and complex human instructions far better than traditional keyword-based systems.
- **Zero-Shot Intelligence:** They can perform tasks they weren't explicitly trained for such as summarizing a document or writing code just by being asked.
- **Information Synthesis:** They don't just "find" text; they **summarize, paraphrase, and connect** disparate ideas into a coherent answer.
- **Complex Reasoning:** LLMs can follow Chain-of-Thought logic to solve multi-step problems, making them excellent processors of information.



From “just” LLMs to RAG

- LLMs Confidently Hallucinate.
- Fine-tuning for specific jobs are costly.
- Opaque Source Attribution
- The knowledge is stuck into the internal model weights



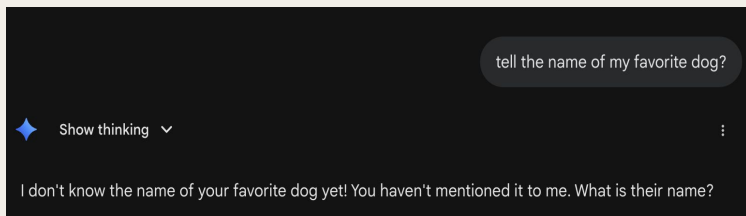
From “just” LLMs to RAG

- LLMs Confidently Hallucinate.
- Fine-tuning for specific jobs are costly.
- Opaque Source Attribution
- The knowledge is stuck into the internal model weights

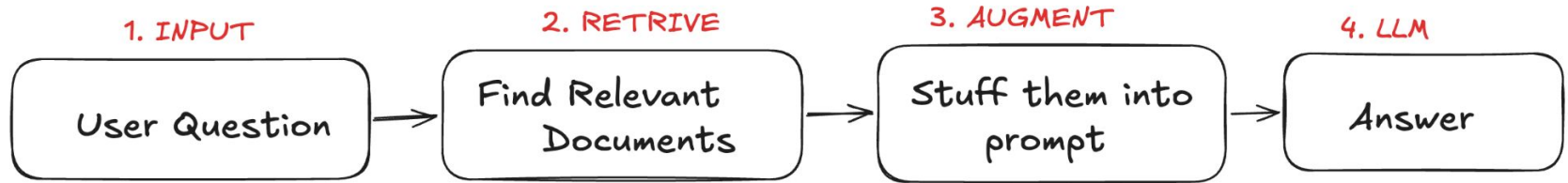
Retrieval Based Language Model:

A language Model that uses an external datastore during Test time.

Instead of asking the model to remember everything, we let it **look things up first, then answer .**

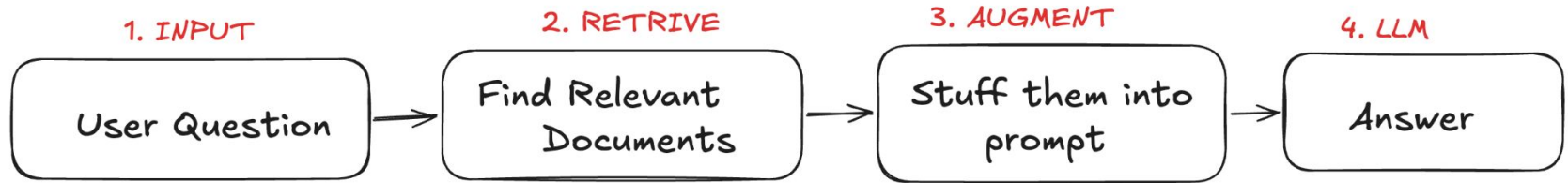


Basic RAG Flow:



BASIC RAG FLOW : Search -> Augment Prompt -> Generate

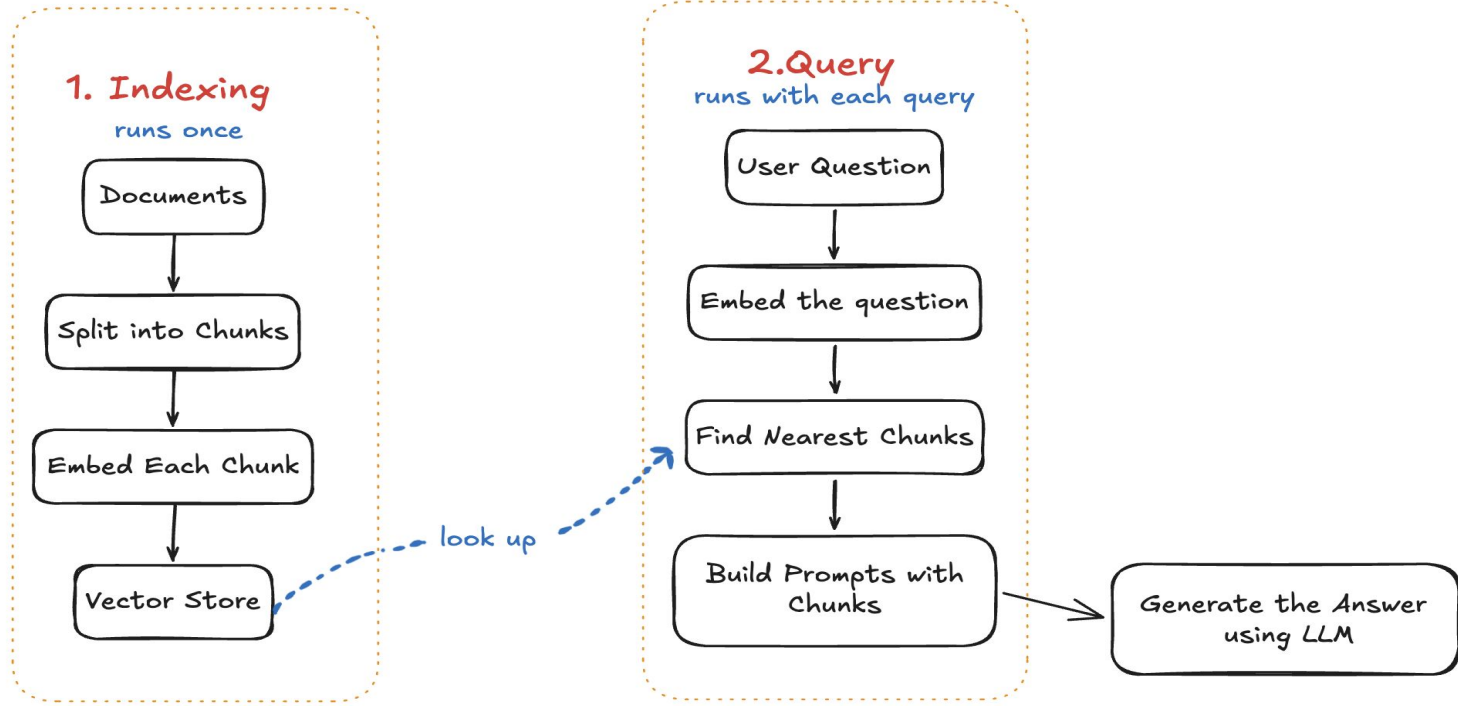
Basic RAG Flow:



BASIC RAG FLOW : Search -> Augment Prompt -> Generate

- Everything else - embeddings, chunking, vector stores - are just making “find relevant documents” smarter

Full RAG pipeline overview:

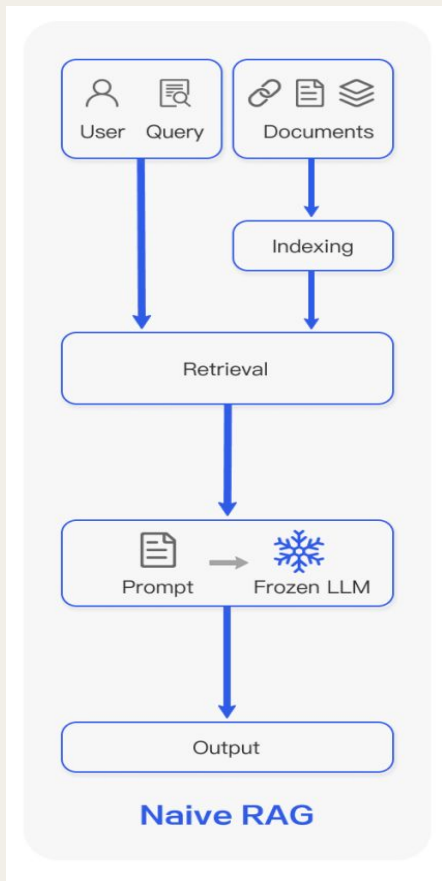


Naive RAG

The Workflow: Follows a simple, linear three-step process:

Indexing → Retrieval → Generation . (Retrieve-Read)

- **Indexing:** Raw data (Docs, Texts) is cleaned, segmented into fixed-size chunks, and converted into vector embeddings.
- **Retrieval:** Uses semantic similarity (like Cosine Similarity) to find the Top-K document chunks most relevant to the user query.
- **Generation:** The LLM reads the retrieved chunks and the original question to synthesize a grounded answer.



Img from: Retrieval-Augmented Generation for Large Language Models: A Survey



Naive RAG

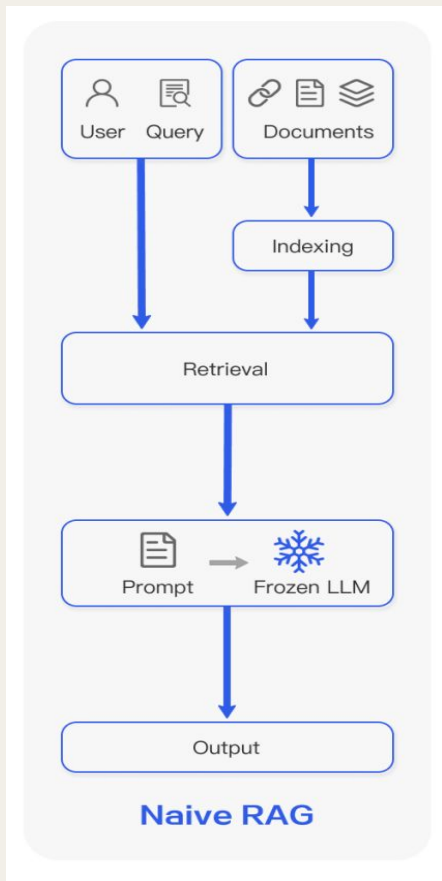
The Workflow: Follows a simple, linear three-step process:

Indexing → Retrieval → Generation . (Retrieve-Read)

- **Indexing:** Raw data (Docs, Texts) is cleaned, segmented into fixed-size chunks, and converted into vector embeddings.
- **Retrieval:** Uses semantic similarity (like Cosine Similarity) to find the Top-K document chunks most relevant to the user query.
- **Generation:** The LLM reads the retrieved chunks and the original question to synthesize a grounded answer.

Drawbacks:

- **Retrieval Challenges:** Low precision such as irrelevant chunks and low recall such as missing key info.
- **Generation Difficulties:** Hallucination where the model ignores the context, or Irrelevance where the answer doesn't match the query.
- **Augmentation Hurdles:** Redundant info from multiple sources leading to repetitive or disjointed answers.



Img from: Retrieval-Augmented Generation for Large Language Models: A Survey



Advanced RAG

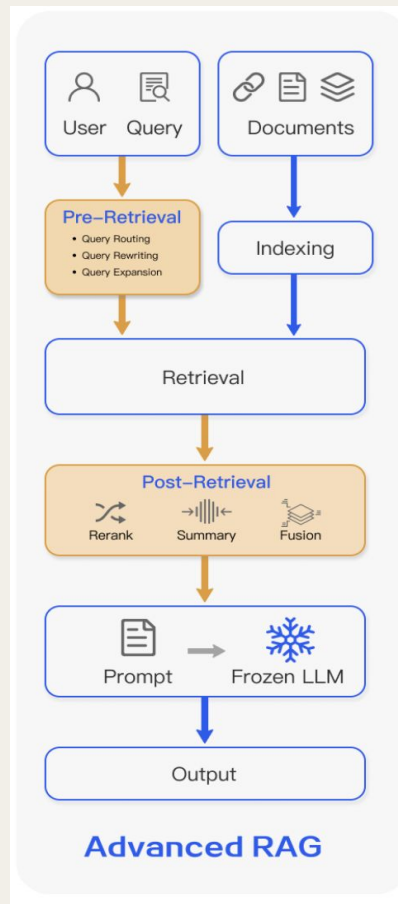
The Goal: Moving beyond Retrieve-Read by adding **Pre-Retrieval** and **Post-Retrieval** optimization.

Pre-Retrieval Strategies (Better Search):

- **Query Rewriting/Transformation:** Using the LLM to rephrase a vague user question into a better search query.
- **Indexing Refinement:** Using a **Sliding Window** or metadata to ensure context isn't cut off mid-sentence.

Post-Retrieval Strategies (Better Context):

- **Reranking:** Re-ordering chunks so the most relevant evidence is at the “edges” of the prompt.
- **Context Compression:** Condensing long chunks into essential facts to reduce noise and lower token costs.



Advanced RAG

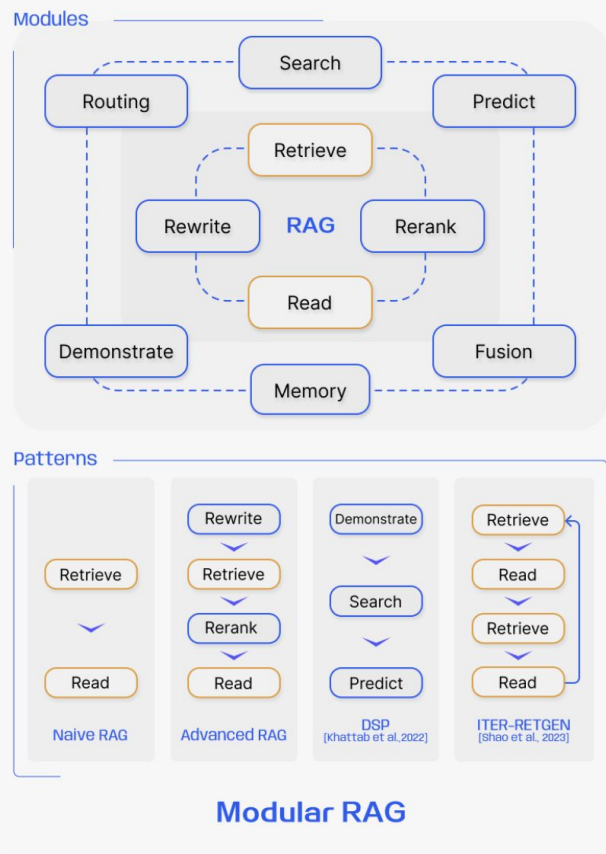
Modular RAG

Main Idea:

- Moving away from a fixed linear pipeline to a flexible, component-based architecture where modules can be rearranged or substituted

New Functional Modules:

- **Search Module:** Allows the system to search across multiple sources simultaneously (e.g., Web, Knowledge Graphs, and Vector Databases).
- **Memory Module:** Uses the LLM's own past iterations to guide current retrieval, creating an "unbounded" memory pool.
- **Adaptive Retrieval:** Automatically tailors the retrieval process to the specific task (e.g., code generation vs. medical QA).



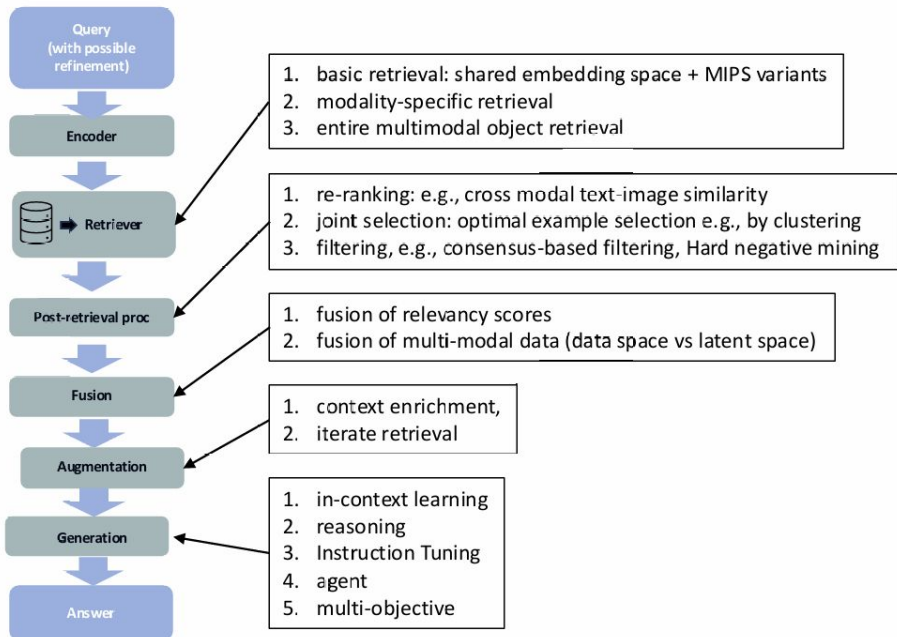
Img from: Retrieval-Augmented Generation for Large Language Models: A Survey

Multimodal RAG: Overview

Example Inputs:

- Text
- Images
- Videos
- Audio

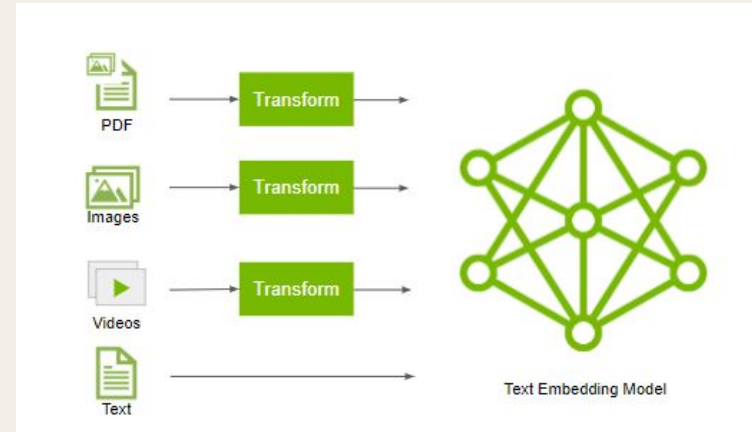
Architecture of Multimodal RAG



Img from: Multimodal Retrieval-Augmented Generation

Multimodal RAG: Retrieval

- **Basic retrieval: Text only**
 - Extracts text descriptions from videos/images (captions, tags)
 - Treats all modalities as text documents
- **Modality-specific retrieval**
 - Different encoders for each mode (text, image, audio)
 - Not good for comparing across modes
- **Entire multimodal object retrieval**
 - Processes multimodal items as one object
 - Maps different modes to the same embedding space

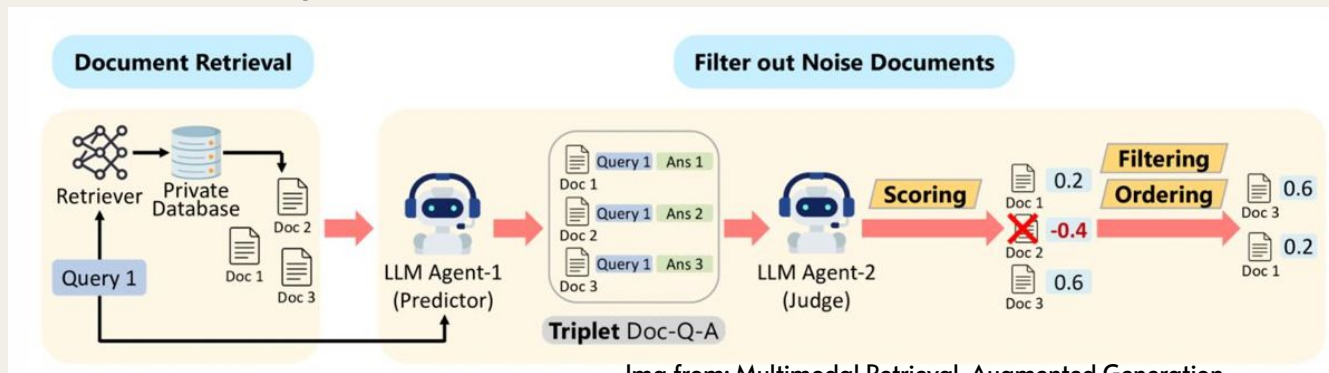


Img from:
<https://bhavishyapandit9.substack.com/p/multimodal-retrieval-augmented-generation>



Multimodal RAG: Post-Retrieval Proc

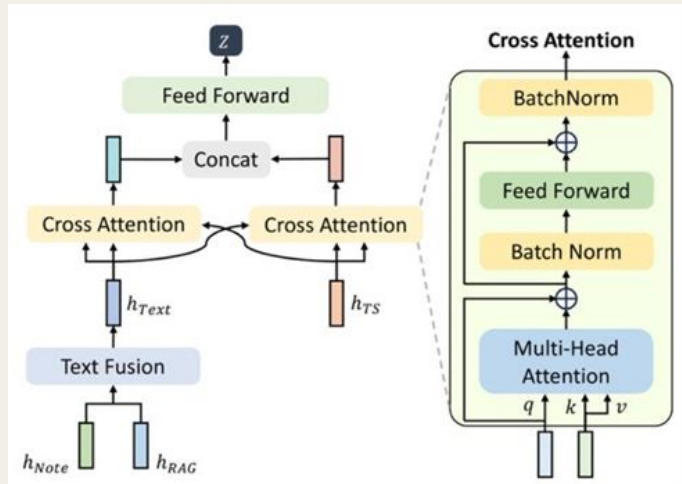
- **Reranking** (scores from different modes)
 - Assumption: Initial retrieval is noisy
 - Rescore top k results with more powerful model
- **Joint Selection**
 - Optimizes combination of different modalities for better context
 - Assumption: Different modes support each other
- **Filtering**
 - Removes noisy and unimportant data chunks



Img from: Multimodal Retrieval-Augmented Generation

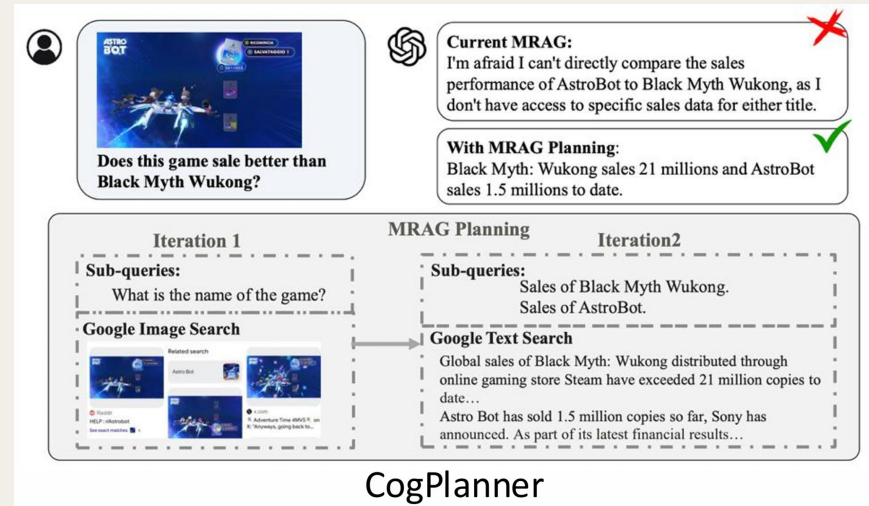
Multimodal RAG: Fusion

- **Relevance Score Fusion:**
 - Combine scores from multiple modalities
- **Multimodal Data Fusion**
 - Data Space: Data fused before embedding (ex. tables, images to unified text format)
 - Latent Space: Embed each mode differently before fusing said embeddings



Multimodal RAG: Augmentation

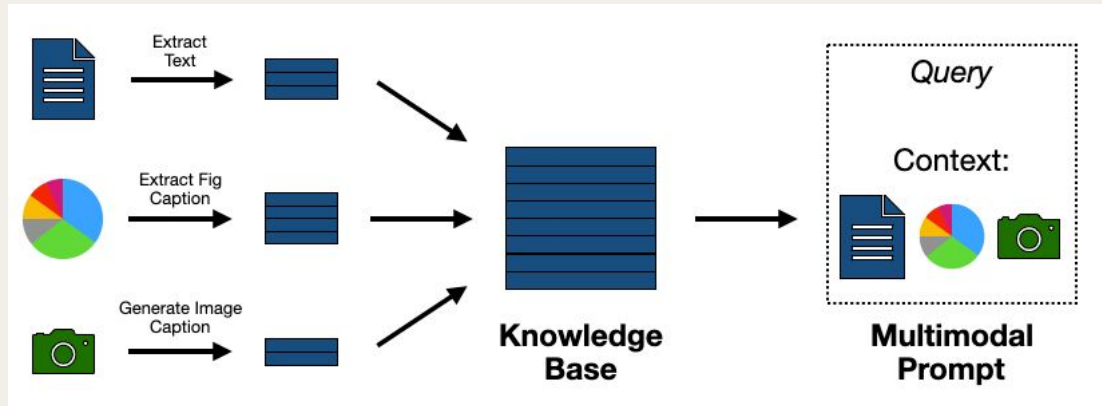
- Context Enrichment
 - Retrieved knowledge is added to the prompt
- Iterate Retrieval
 - Retrieval is run multiple times during LLM reasoning (once isn't enough)
 - Supports multiseq/complex tasks



Img from: Multimodal Retrieval-Augmented Generation

Multimodal RAG: Generation

- Multimodal RAG helps with:
 - Incontext Learning
 - Reasoning
 - Instruction Tuning
 - Agentic
 - Multiobjective Generation

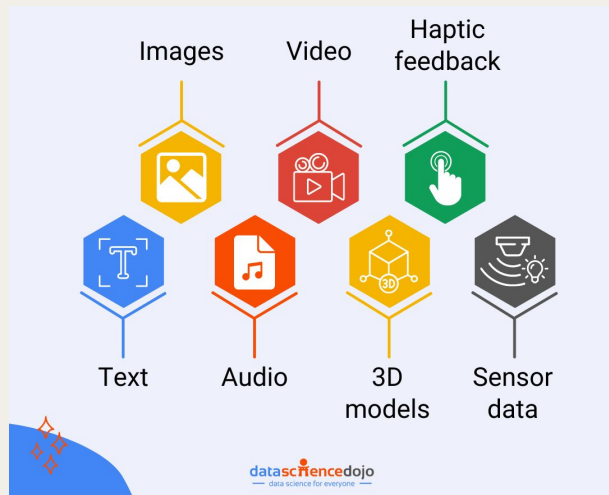


Img from:
<https://towardsdatascience.com/multimodal-rag-process-any-file-type-with-ai-e6921342c903/>



Multimodal RAG: Challenges

- **Modality Alignment**
 - Connecting information across modalities
 - Different modalities may represent concepts differently
- **High computation cost**
 - Processing audio, images, text, videos together is expensive
 - Multiple encoders
 - High memory and latency
- **Evaluation Metrics**
 - Existing benchmarks are for single-modalities
 - Difficult to measure cross modal reasoning quality



Break

Retrieval-based LM: Architecture

I. What to retrieve?

- **Text chunks or passages:** The most common approach, used by models like REALM and RETRO. While datastores of text chunks are space-efficient, they require more computation.
- **Tokens:** Used by models like kNN-LM, retrieving individual tokens is more fine-grained and compute-efficient, but storing them requires significantly more space.
- **Entities or entity mentions:** Retrieving specific entities is space-efficient and highly effective for entity-centric tasks, though it requires an additional entity detection step.
- **Its own input:** Models can retrieve from their own past input context, which is an effective strategy for long-sequence modeling.

Retrieval-based LM: Architecture

2. How to use the retrieved data?

- **Input layer:** The retrieved text is simply concatenated to the input prompt (e.g., Retrieve-in-context LMs). This is simpler to implement but can be slower.
- **Intermediate layers:** The retrieved data is woven into the internal layers of the neural network using techniques like "Chunked Cross Attention" (e.g., RETRO). This requires complex training but can be designed to be highly efficient.
- **Output layer:** The retrieved data acts as a final filter to adjust the probability of the words the language model outputs (e.g., kNN-LM).

Retrieval-based LM: Architecture

3. When to retrieve?

- **Once:** Retrieving only a single time at the beginning of the prompt.
- **Frequently (Every n tokens or Every token):** Retrieving more frequently generally leads to better performance, but it comes at the cost of slower inference times.
- **Adaptively:** Models like FLARE use internal triggers to only retrieve data when the model is "unsure" of the answer, saving computing power while maintaining accuracy.

Few Architectures :

REALM is a classic example of standard retrieval architecture.

- **What it retrieves:** Full text chunks (passages).
- **When it retrieves:**
Typically just once, at the very beginning of the query.
- **How it uses it:** It feeds the data into the **input layer** by simply concatenating the retrieved text to the user's prompt.

Few Architectures :

REALM is a classic example of standard retrieval architecture.

- **What it retrieves:** Full text chunks (passages).
- **When it retrieves:** Typically just once, at the very beginning of the query.
- **How it uses it:** It feeds the data into the **input layer** by simply concatenating the retrieved text to the user's prompt.

RETRO was designed to solve the inefficiency of stuffing millions of words into the input layer.

- **What it retrieves:** Full text chunks.
- **When it retrieves:** Frequently (every n tokens).
- **How it uses it:** It injects the retrieved data directly into the **intermediate layers** (the internal "hidden" layers) of the neural network.

Few Architectures :

REALM is a classic example of standard retrieval architecture.

- **What it retrieves:** Full text chunks (passages).
- **When it retrieves:** Typically just once, at the very beginning of the query.
- **How it uses it:** It feeds the data into the **input layer** by simply concatenating the retrieved text to the user's prompt.

RETRO was designed to solve the inefficiency of stuffing millions of words into the input layer.

- **What it retrieves:** Full text chunks.
- **When it retrieves:** Frequently (every n tokens).
- **How it uses it:** It injects the retrieved data directly into the **intermediate layers** (the internal "hidden" layers) of the neural network.

kNN-LM (k-Nearest Neighbors Language Model) takes a radically different, highly granular approach.

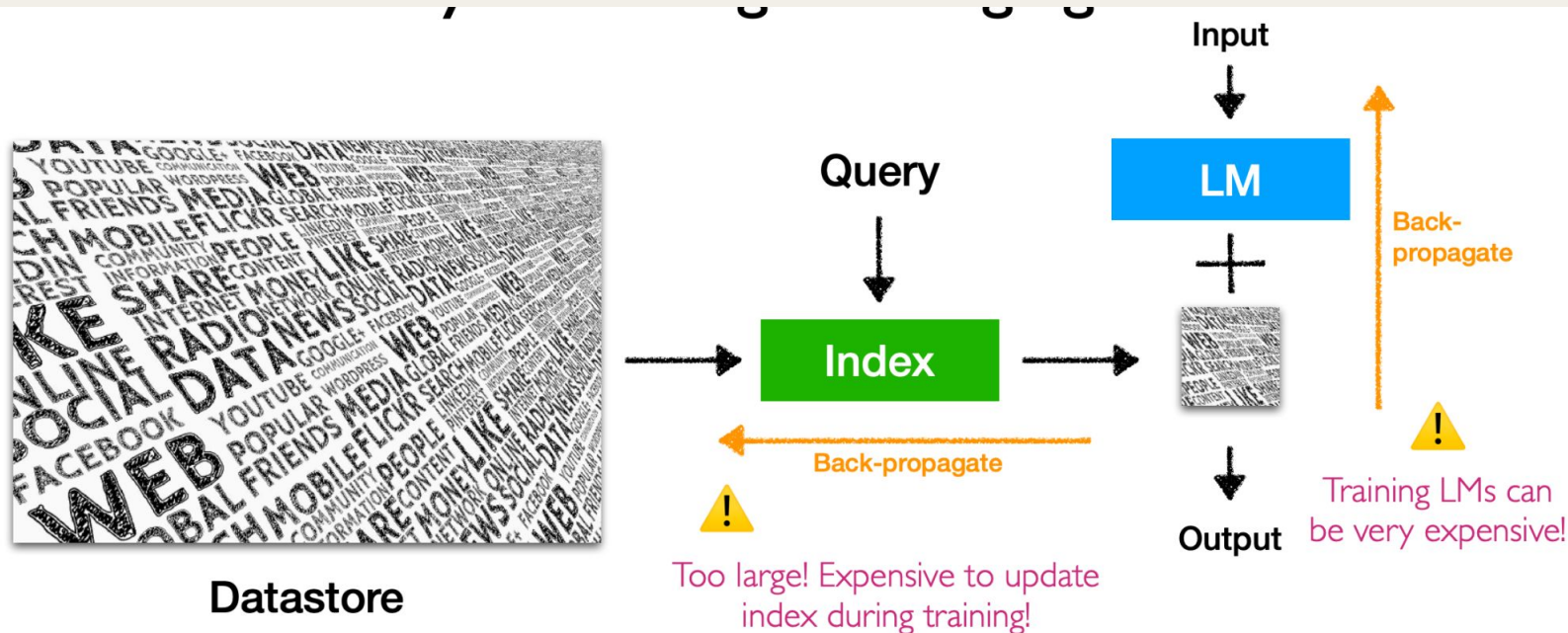
- **What it retrieves:** Individual tokens, rather than full text passages.
- **When it retrieves:** Constantly—for every single token the model generates.
- **How it uses it:** It applies the retrieval at the final **output layer**.

Retrieval based LMs Training: RALM

- The Researcher : RETRIEVAL MODEL that finds the documents
- The Writer : LARGE LANGUAGE MODEL that reads those documents and pens the answer



Retrieval based LMs Training: Challenges



Retrieval based LMs Training: Types

- Independent Training
- Sequential Training
- Joint Training (with asynchronous index update)

Independent Training: Ram et al. 2023

- Used “off the shelves” LM which did not require pre-training
- Focussed on how to choose documents for improved LLM performance

Retrieval Stride While in the above formulation a retrieval operation can occur at each generation step, we might want to perform retrieval only once every $s > 1$ tokens due to the cost of calling the retriever, and the need to replace the documents in the LM prefix during generation. We refer to s as the *retrieval stride*. This gives rise to the following In-Context RALM formulation (which reduces back to Eq. (2) for $s = 1$):

$$p(x_1, \dots, x_n) = \prod_{j=0}^{n_s-1} \prod_{i=1}^s p_{\theta}(x_{s \cdot j+i} | [\mathcal{R}_C(x_{\leq s \cdot j}); x_{<(s \cdot j+i)}]) , \quad (3)$$

where $n_s = n/s$ is the number of retrieval strides.

Independent Training: Ram et al. 2023

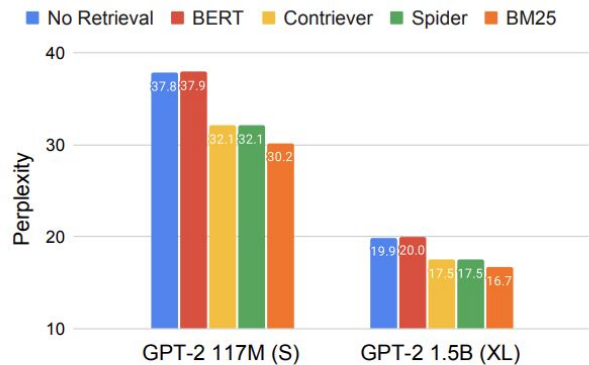


Figure 3: The performance of four off-the-shelf retrievers used for In-Context RALM on the development set of WikiText-103. All RALMs are run with $s = 4$ (i.e., retrieval is applied every four tokens). For each RALM, we report the result of the best query length ℓ (see Figures 6, 9, 10).

Model	Retrieval	Reranking	WikiText-103	RealNews	ArXiv	Stack Exch.	FreeLaw
			word ppl	token ppl	token ppl	token ppl	token ppl
GPT-2 S	–	–	37.5	21.3	12.0	12.8	13.0
	BM25 §5	–	29.6	16.1	10.9	11.3	9.6
	BM25	Zero-shot §6.1	28.6	15.5	10.1	10.6	8.8
	BM25	Predictive §6.2	26.8	–	–	–	–
GPT-2 M	–	–	26.3	15.7	9.3	8.8	9.6
	BM25 §5	–	21.5	12.4	8.6	8.1	7.4
	BM25	Zero-shot §6.1	20.8	12.0	8.0	7.7	6.9
	BM25	Predictive §6.2	19.7	–	–	–	–
GPT-2 L	–	–	22.0	13.6	8.4	8.5	8.7
	BM25 §5	–	18.1	10.9	7.8	7.8	6.8
	BM25	Zero-shot §6.1	17.6	10.6	7.3	7.4	6.4
	BM25	Predictive §6.2	16.6	–	–	–	–
GPT-2 XL	–	–	20.0	12.4	7.8	8.0	8.0
	BM25 §5	–	16.6	10.1	7.2	7.4	6.4
	BM25	Zero-shot §6.1	16.1	9.8	6.8	7.1	6.0
	BM25	Predictive §6.2	15.4	–	–	–	–

Table 1: Perplexity on the test set of WikiText-103, RealNews and three datasets from the Pile. For each LM, we report: (a) its performance without retrieval, (b) its performance when fed the top-scored passage by BM25 (§5), and (c) its performance when applied on the top-scored passage of each of our two suggested rerankers (§6). All models share the same vocabulary, thus token-level perplexity (*token ppl*) numbers are comparable. For WikiText we follow prior work and report word-level perplexity (*word ppl*).

Better Retrieval models + Better LLMs = Better Retrieval Based LLMs

Sequential Training: Shi et al. 2023

REPLUG: Retrieval-Augmented Black-Box Language Models

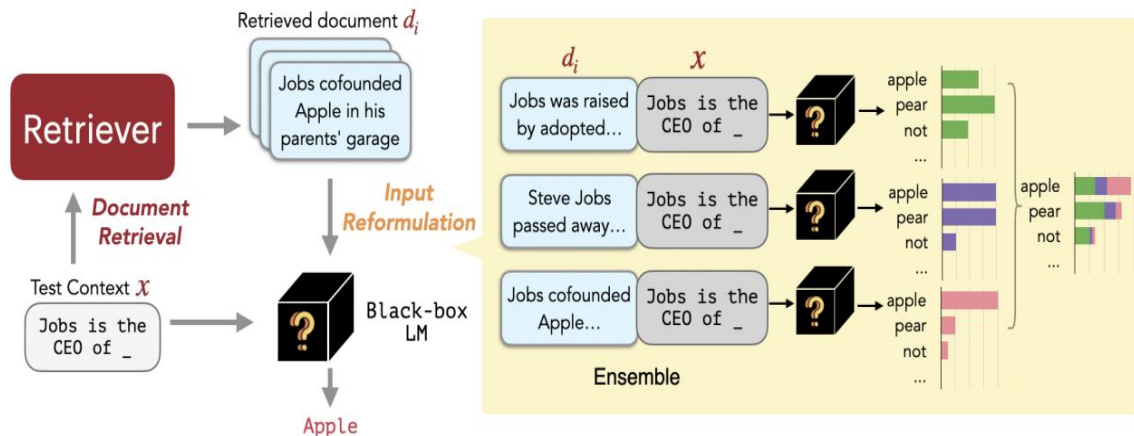


Figure 2. REPLUG at inference (§3). Given an input context, REPLUG first retrieves a small set of relevant documents from an external corpus using a retriever (§3.1 *Document Retrieval*). Then it prepends each document separately to the input context and ensembles output probabilities from different passes (§3.2 *Input Reformulation*).

Sequential Training: Shi et al. 2023

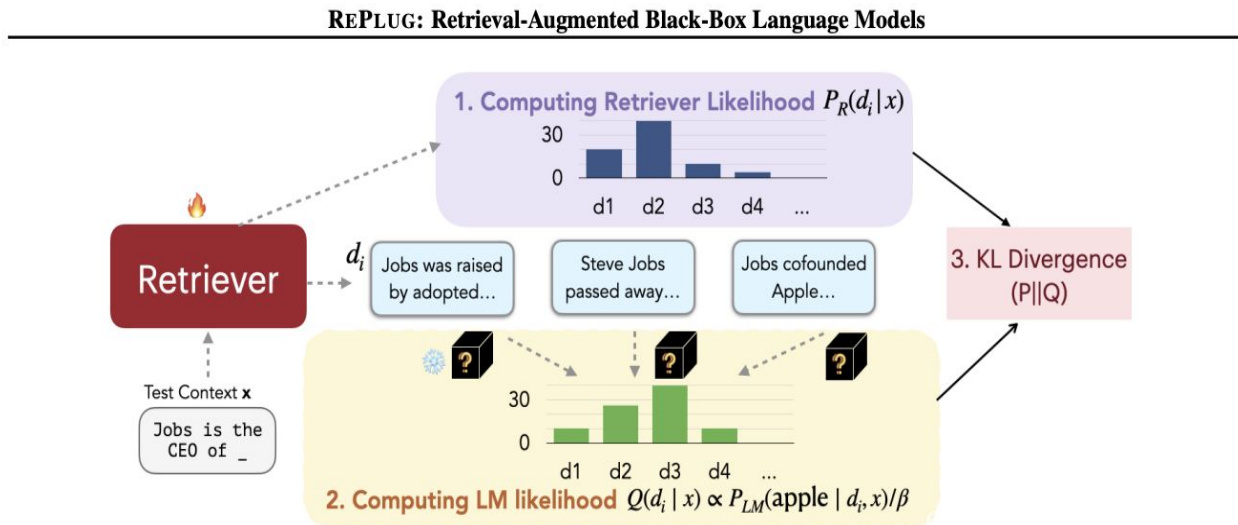
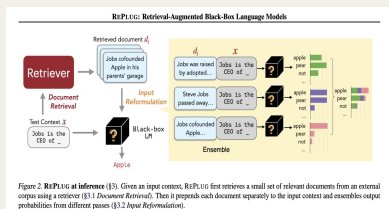
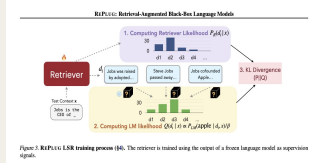
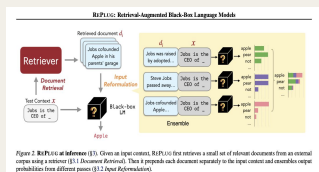


Figure 3. **REPLUG LSR training process (§4)**. The retriever is trained using the output of a frozen language model as supervision signals.

Sequential Training: Shi et al. 2023



REPLUG: Retrieval-Augmented Black-Box Language Models

Model		# Parameters	Original	+ REPLUG	Gain %	+ REPLUG LSR	Gain %
GPT-2	Small	117M	1.33	1.26	5.3	1.21	9.0
	Medium	345M	1.20	1.14	5.0	1.11	7.5
	Large	774M	1.19	1.15	3.4	1.09	8.4
	XL	1.5B	1.16	1.09	6.0	1.07	7.8
GPT-3 (black-box)	Ada	350M	1.05	0.98	6.7	0.96	8.6
	Babbage	1.3B	0.95	0.90	5.3	0.88	7.4
	Curie	6.7B	0.88	0.85	3.4	0.82	6.8
	Davinci	175B	0.80	0.77	3.8	0.75	6.3

Table 1. Both REPLUG and REPLUG LSR consistently enhanced the performance of different language models. Bits per byte (BPB) of the Pile using GPT-3 and GPT-2 family models (Original) and their retrieval-augmented versions (+REPLUG and +REPLUG LSR). The gain % shows the relative improvement of our models compared to the original language model.

Model	# Parameters	Humanities	Social.	STEM	Other	All
Codex	175B	74.2	76.9	57.8	70.1	68.3
PaLM	540B	77.0	81.0	55.6	69.6	69.3
Flan-PaLM	540B	-	-	-	-	72.2
Atlas	11B	46.1	54.6	38.8	52.8	47.9
Codex + REPLUG	175B	76.0	79.7	58.8	72.1	71.4
Codex + REPLUG LSR	175B	76.5	79.9	58.9	73.2	71.8

Table 2. REPLUG and REPLUG LSR improves Codex by 4.5% and 5.1% respectively. Performance on MMLU broken down into 4 categories. The last column averages the performance over these categories. All models are evaluated based on 5-shot in-context learning with direct prompting.

Joint Training: Gu et al. 2020

REALM decomposes $p(y | x)$ into two steps: *retrieve*, then *predict*. Given an input x , we first retrieve possibly helpful documents z from a knowledge corpus $\mathcal{Z}$. We model this as a sample from the distribution $p(z | x)$. Then, we condition on both the retrieved z and the original input x to generate the output y —modeled as $p(y | z, x)$. To obtain the overall likelihood of generating y , we treat z as a latent variable and marginalize over all possible documents z , yielding

$$p(y | x) = \sum_{z \in \mathcal{Z}} p(y | z, x) p(z | x). \quad (1)$$

Joint Training: Gu et al. 2020

REALM decomposes $p(y | x)$ into two steps: *retrieve*, then *predict*. Given an input x , we first retrieve possibly helpful documents z from a knowledge corpus $\mathcal{Z}$. We model this as a sample from the distribution $p(z | x)$. Then, we condition on both the retrieved z and the original input x to generate the output y —modeled as $p(y | z, x)$. To obtain the overall likelihood of generating y , we treat z as a latent variable and marginalize over all possible documents z , yielding

$$p(y | x) = \sum_{z \in \mathcal{Z}} p(y | z, x) p(z | x). \quad (1)$$

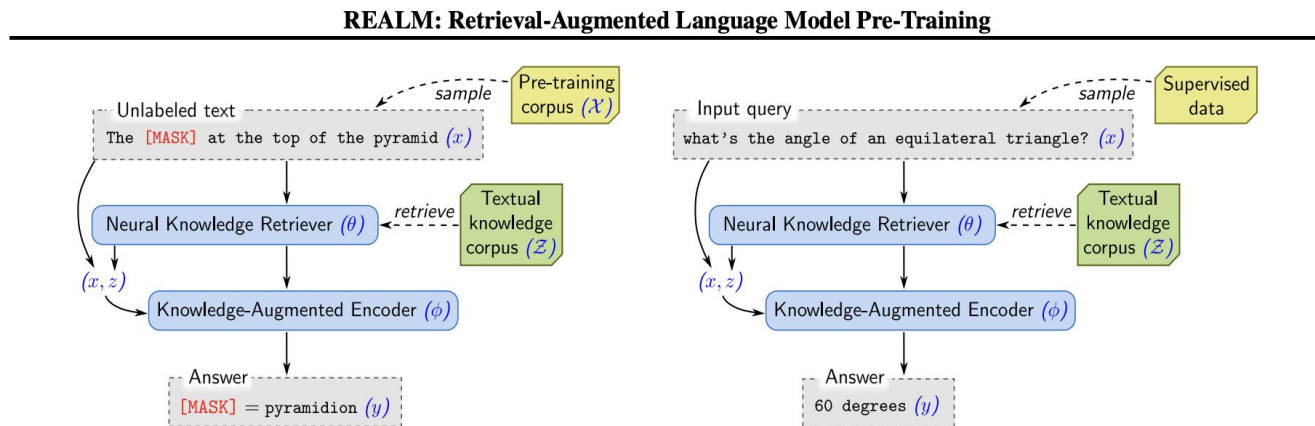


Figure 2. The overall framework of REALM. **Left:** *Unsupervised pre-training*. The knowledge retriever and knowledge-augmented encoder are jointly pre-trained on the unsupervised language modeling task. **Right:** *Supervised fine-tuning*. After the parameters of the retriever (θ) and encoder (ϕ) have been pre-trained, they are then fine-tuned on a task of primary interest, using supervised examples.

Joint Training: Gu et al. 2020

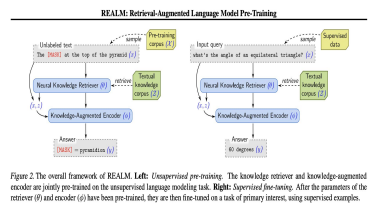
REALM decomposes $p(y|x)$ into two steps: *retrieve*, then *predict*. Given an input x , we first retrieve possibly helpful documents z from a knowledge corpus $\mathcal{Z}$. We model this as a sample from the distribution $p(z|x)$. Then, we condition on both the retrieved z and the original input x to generate the output y —modeled as $p(y|x, z)$. To obtain the overall likelihood of generating y , we treat z as a latent variable and marginalize over all possible documents z , yielding

$$p(y|x) \approx \sum_{z \in \mathcal{Z}} p(z|x, x) p(y|x, z). \quad (1)$$

REALM: Retrieval-Augmented Language Model Pre-Training

Table 1. Test results on Open-QA benchmarks. The number of train/test examples are shown in parentheses below each benchmark. Predictions are evaluated with exact match against any reference answer. Sparse retrieval denotes methods that use sparse features such as TF-IDF and BM25. Our model, REALM, outperforms all existing systems.

Name	Architectures	Pre-training	NQ (79k/4k)	WQ (3k/2k)	CT (1k /1k)	# params
BERT-Baseline (Lee et al., 2019)	Sparse Retr.+Transformer	BERT	26.5	17.7	21.3	110m
T5 (base) (Roberts et al., 2020)	Transformer Seq2Seq	T5 (Multitask)	27.0	29.1	-	223m
T5 (large) (Roberts et al., 2020)	Transformer Seq2Seq	T5 (Multitask)	29.8	32.2	-	738m
T5 (11b) (Roberts et al., 2020)	Transformer Seq2Seq	T5 (Multitask)	34.5	37.4	-	11318m
DrQA (Chen et al., 2017)	Sparse Retr.+DocReader	N/A	-	20.7	25.7	34m
HardEM (Min et al., 2019a)	Sparse Retr.+Transformer	BERT	28.1	-	-	110m
GraphRetriever (Min et al., 2019b)	GraphRetriever+Transformer	BERT	31.8	31.6	-	110m
PathRetriever (Asai et al., 2019)	PathRetriever+Transformer	MLM	32.6	-	-	110m
ORQA (Lee et al., 2019)	Dense Retr.+Transformer	ICT+BERT	33.3	36.4	30.1	330m
Ours ($\mathcal{X}$ = Wikipedia, $\mathcal{Z}$ = Wikipedia)	Dense Retr.+Transformer	REALM	39.2	40.2	46.8	330m
Ours ($\mathcal{X}$ = CC-News, $\mathcal{Z}$ = Wikipedia)	Dense Retr.+Transformer	REALM	40.4	40.7	42.9	330m



When to use Retrieval based LMs

- **Long-tail knowledge:** Standard LMs struggle to remember obscure or infrequent facts, and simply making the LM larger does not easily solve this. Retrieval systems easily solve this by finding specific documents for long-tail queries, leading to large performance gains.
- **Knowledge update:** World knowledge changes (e.g., the population of a city changes over time). Instead of expensively retraining the entire LM to learn these new facts, you can simply update the retrieval "datastore" with new documents, and the model will instantly use the updated information.
- **Parameter-efficiency:** By pairing a small language model with a retrieval system, it can often outperform massive, highly expensive models (like vanilla GPT-3) on factual tests.
- **Verifiability:** Because the model pulls from specific documents, it can provide exact attributions or quotes. This allows humans to easily verify if the model's answer is factually supported by evidence.

Results:

Doc Prompting : Zhu et al. 2023

Table 3: Results on CoNaLa, using a CodeT5 retriever with top-10 retrieved docs. Function recall (Recall) measures how many functions in the reference code are correctly predicted, and unseen function recall (Recall_{unseen}) only considers the subset held out from the training data.

Model		BLEU	Recall	Recall _{unseen}
Codex 3-shots	-	43.16	39.52	-
	+ DocPrompting	43.47	39.87	-
	+ DocPrompting oracle docs	50.59	57.84	-
T5	-	28.07	14.36	2.57
	+ DocPrompting	30.04	21.34	8.24
CodeT5	-	34.57	24.24	9.03
	+ DocPrompting	36.22	27.80	18.30
	+ DocPrompting oracle docs	49.04	72.20	63.91

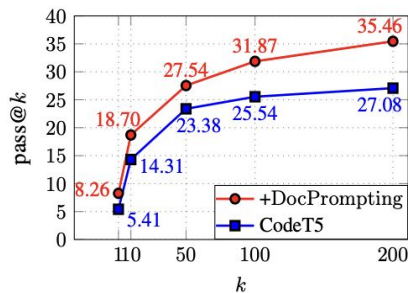


Figure 3: Pass@k of CodeT5 with and without DocPrompting on 100 CoNaLa examples.

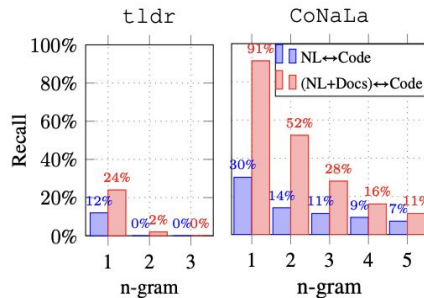


Figure 4: Using documentation significantly increases the n -gram overlap recall between the input and the output, in tldr and CoNaLa.

Results:

Table 3: Results on CoNaLa, using a CodeT5 retriever with top-10 retrieved docs. Function recall (Recall) measures how many functions in the reference code are correctly predicted, and unseen function recall (Recall_{unseen}) only considers the subset held out from the training data.

Model	BLEU	Recall	Recall _{unseen}
Codex J-aiots	-	43.16	39.52
+ DocPrompting	43.47	39.87	-
+ DocPrompting oracle docs	50.59	57.84	-
T5	-	28.07	14.36
+ DocPrompting	36.04	21.34	8.24
CodeT5	-	34.57	24.24
+ DocPrompting	36.22	27.80	18.30
+ DocPrompting oracle docs	49.04	72.20	63.91

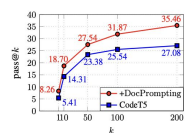


Figure 3: Pass@k of CodeT5 with and without DocPrompting on 100 CoNaLa examples.

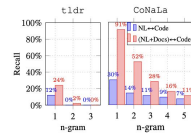
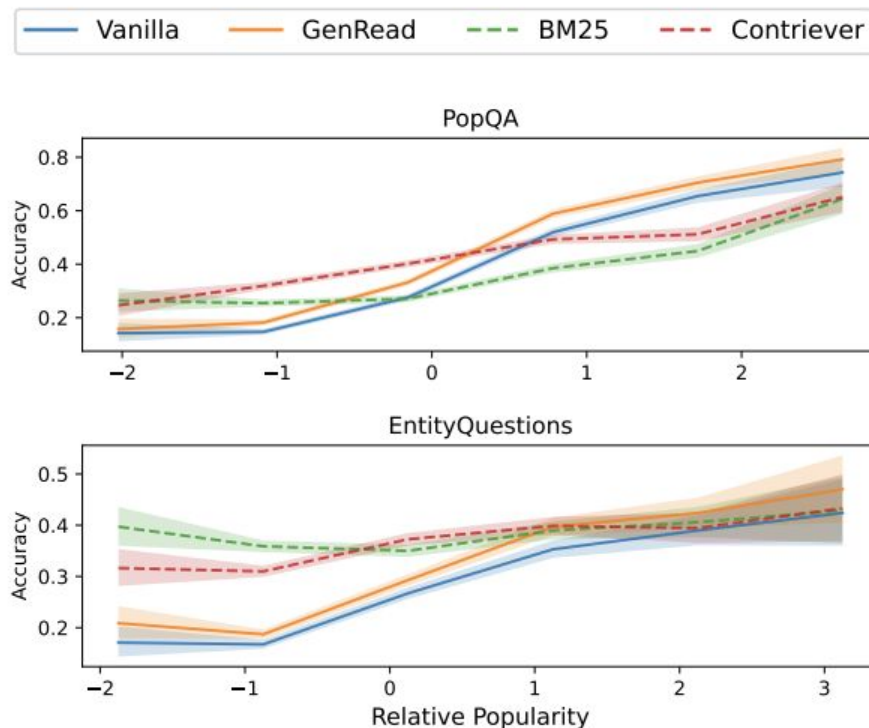


Figure 4: Using documentation significantly increases the n-gram overlap recall between the input and the output, in tldr and CoNaLa.

Mallen , Asai et al. 2023: When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories



Results:

Table 3: Results on CoSQuAD using a GPT-3.5-turbo with top-10 retrieved docs. Precision recall (Recall) measures how many functions in the inference code are correctly predicted, and answer function recall (Recall_{answer}) only consider the subset held out from the training data.

Model	BLEU	Recall	Recall _{answer}
Codex J-001	43.39	9.23	-
+ DocPrompting	45.47	20.87	-
+ DocPrompting each doc	50.39	27.84	-
T5	58.87	14.34	2.57
+ DocPrompting	59.64	23.54	8.24
Gpt3.5	54.37	24.24	9.43
+ DocPrompting	56.42	27.88	16.36
+ DocPrompting each doc	49.84	32.33	63.91

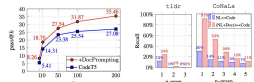


Figure 3: Precision-Recall curves for CodexT5 with and without DocPrompting on 100 CoSQuAD samples.

Figure 4: Using decomposition significantly decreases the sign overlap recall between the type and the output. In CoSQuAD and CoSQuAD.

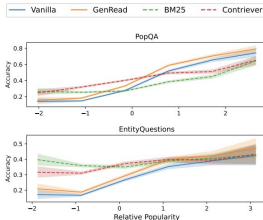


Table 7: **Comparison to state-of-the-art on MMLU.** *For the 5-shot setting, ATLAS uses fine-tuning, while previous works use in-context learning. The ATLAS model uses de-biased inference. Train FLOPS refers to total the amount of computation necessary to train the model, including pre-training and/or fine-tuning.

Setting	Model	Params	Train FLOPS	All	Hum.	Soc. Sci.	STEM	Other
zero-shot	ATLAS	11B	3.5e22	47.1	43.6	54.1	38.0	54.4
5-shot	GPT-3	175B	3.1e23	43.9	40.8	50.4	36.7	48.8
	Gopher	280B	5.0e23	60.0	56.2	71.9	47.4	66.1
	Chinchilla	70B	5.0e23	67.5	63.6	79.3	55.0	73.9
	ATLAS*	11B	3.5e22	47.9	46.1	54.6	38.8	52.8
5-shot (multi-task)	ATLAS	11B	3.5e22	56.6	50.1	66.4	46.4	66.2
Full / Transfer	UnifiedQA	11B	3.3e22	48.9	45.6	56.6	40.2	54.6
	GPT-3	175B	3.1e23	53.9	52.5	63.9	41.4	57.9
	ATLAS	11B	3.5e22	66.0	61.1	77.2	53.2	74.4

Table 8: **Comparison to state-of-the-art on question answering.** We report results on NaturalQuestions, and on TriviaQA for both the filtered set, commonly used for open-domain question answering and the unfiltered hidden set for which evaluation is accessible online: <https://competitions.codalab.org/competitions/17208>. For the 64-shot setting, our model uses fine-tuning, while the other models use prompting.

Model	NQ		TriviaQA filtered		TriviaQA unfiltered	
	64-shot	Full	64-shot	Full	64-shot	Full
GPT-3 (Brown et al., 2020)	29.9	-	-	-	71.2	-
Gopher (Rae et al., 2021)	28.2	-	57.2	-	61.3	-
Chinchilla (Hoffmann et al., 2022)	35.5	-	64.6	-	72.3	-
PaLM (Chowdhery et al., 2022)	39.6	-	-	-	81.4	-
RETRO (Borgeaud et al., 2021)	-	45.5	-	-	-	-
FiD (Izcard & Grave, 2020)	-	51.4	-	67.6	-	80.1
FiD-KD (Izcard & Grave, 2021)	-	54.7	-	73.3	-	-
R2-D2 (Fajcik et al., 2021)	-	55.9	-	69.9	-	-
ATLAS	42.4	60.4	74.5	79.8	84.7	89.4

Izcard et al. 2023 : Atlas: Few-shot Learning with Retrieval Augmented Language Models

How to Adapt Retrieval based LMs

- **Fine-Tuning:** Involves actively training and updating the internal weights of the LM, the Retriever, or both using specific task data. Models like ATLAS use this method. While highly effective, it is computationally expensive.
- **Reinforcement Learning with Human Feedback (RLHF):** Systems like GopherCite use human raters to grade the model's output. If the model provides an answer that is both plausible and supported by a real quote from the retrieved document, it is rewarded, teaching the model to prioritize verified facts.
- **Prompting:** This is a simpler method that does not require retraining the LM at all.
 - *Input Space (Retrieval-in-context):* The system fetches a relevant document and pastes the text directly into the prompt alongside the user's question before the LM generates an answer. Eg: REPLUG and Doc Prompting use this method.
 - *Output Space:* The system uses a method like kNN-Prompt, where the retrieved data isn't shown to the LM, but is instead used at the very end to mathematically adjust the probability of the final words the model chooses to output.

Retrieval-Augmented Multimodal Language Modeling from Yasunaga et al. (2022)

RA-CM3: First multimodal model with text and image retrieval and generation

Retriever: Pretrained CLIP

Generator: CM3 transformer trained on the LAION dataset

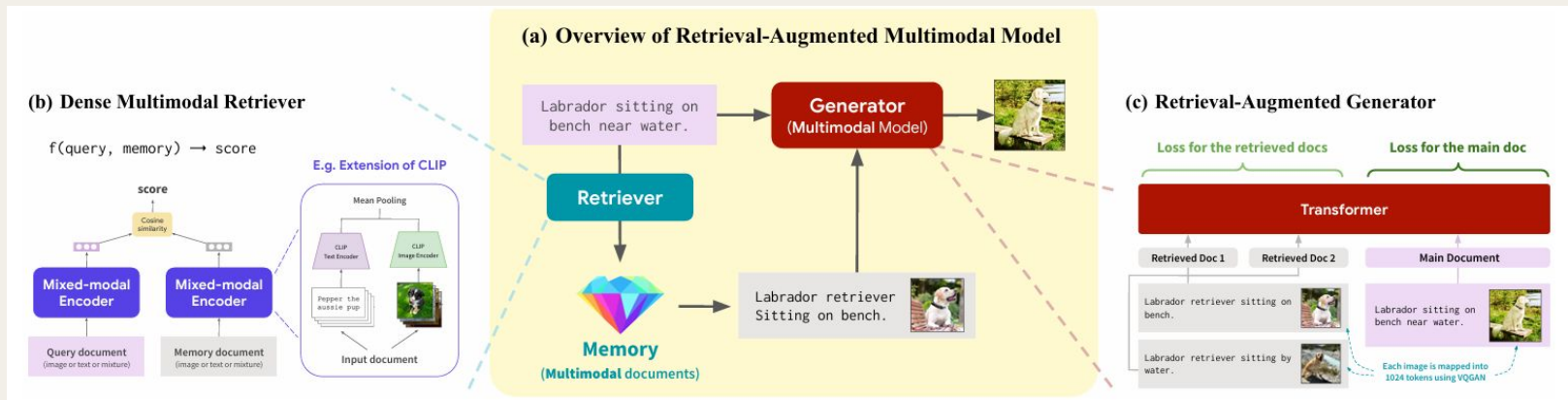


Image from: <https://arxiv.org/pdf/2211.12561>

Multimodal RAG: Results from Yasunaga et al. (2022)

Retrieval-Augmented Multimodal Language Modeling

Approach	Model type	Image generation	Text generation	Retrieval	In-context learning
DALL-E, Parti (Ramesh et al.; Yu et al.)	Autoregressive	✓			
DALL-E 2, Imagen (Ramesh et al.; Saharia et al.)	Diffusion	✓			
Re-Imagen (Chen et al.)	Diffusion	✓		✓	
Flamingo, MetaLM (Alayrac et al.; Hao et al.)	Autoregressive		✓		✓
MuRAG (Chen et al.)	Autoregressive		✓ [†]	✓	
CM3 (Aghajanyan et al.)	Autoregressive	✓	✓		
RA-CM3 (Ours)	Autoregressive	✓	✓	✓	✓

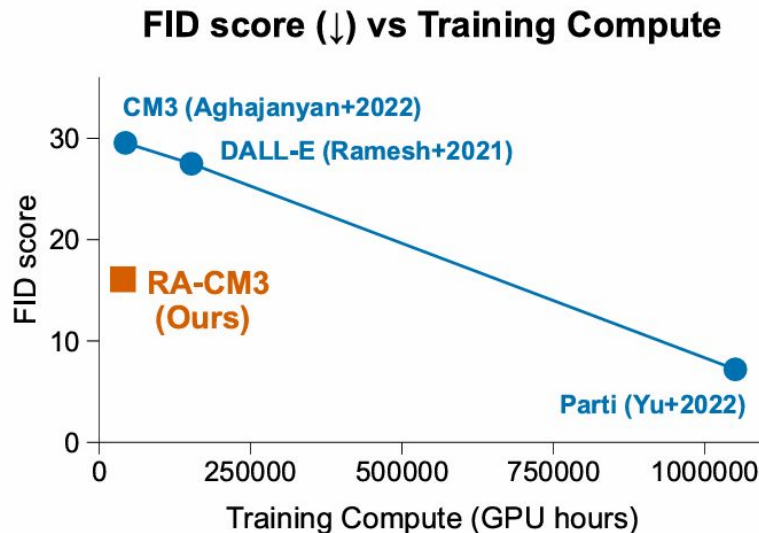
Table 1. **Comparison with other multimodal models.** Our RA-CM3 is the first retrieval-augmented model that can perform both image and text generation. RA-CM3 also exhibits strong in-context learning abilities thanks to the proposed retrieval-augmented training (§3.3).

[†]Focus on question answering.

Multimodal RAG: Results from Yasunaga et al. (2022)

Approach	Model type	MS-COCO FID ($\downarrow$)	
		Not finetuned	Finetuned
Retrieval Baseline	-	17.97	-
KNN-Diffusion (Ashual et al., 2022)	Diffusion	16.66	-
Stable Diffusion (Rombach et al., 2022)	Diffusion	12.63	-
GLIDE (Nichol et al., 2021)	Diffusion	12.24	-
DALL-E 2 (Ramesh et al., 2022)	Diffusion	10.39	-
Imagen (Saharia et al., 2022)	Diffusion	7.27	-
Re-Imagen (Chen et al., 2022b)	Diffusion	6.88	5.25
DALL-E (12B) (Ramesh et al., 2021)	Autoregressive	~28	-
CogView (4B) (Ding et al., 2021)	Autoregressive	27.1	-
CogView2 (6B) (Ding et al., 2022)	Autoregressive	24.0	17.7
Parti (20B) (Yu et al., 2022)	Autoregressive	7.23	3.22
Vanilla CM3	Autoregressive	29.5	-
RA-CM3 (2.7B) (Ours)	Autoregressive	15.7	-

Table 2. **Caption-to-image generation performance on MS-COCO.** Our retrieval-augmented CM3 significantly outperforms the baseline CM3 with no retrieval, as well as other models such as DALL-E (12B parameters). Moreover, our model achieves strong performance with much less training compute than existing models; see Figure 2 for details.



Plot and table from: <https://arxiv.org/pdf/2211.12561>

Multimodal RAG: Results from Yasunaga et al.(2022)

Approach	CIDEr (↑)
Retrieval Baseline	84.1
DALL-E ^{Small} (Wang, 2021)	20.2
ruDALL-E-XL (Forever, 2021)	38.7
minDALL-E (Kim et al., 2021)	48.0
X-LXMERT (Cho et al., 2020)	55.8
Parti (Yu et al., 2022)	83.9
Flamingo (3B; 4-shot) (Alayrac et al., 2022)	85
Flamingo (80B; 4-shot) (Alayrac et al., 2022)	103
Vanilla CM3	71.9
RA-CM3 (2.7B) (Ours)	89.1

Table 3. Image-to-caption generation performance on MS-COCO (with no finetuning). Our retrieval-augmented CM3 significantly outperforms the baseline CM3 with no retrieval. Moreover, our model outperforms other strong models such as Parti (20B parameters) and Flamingo (3B; 4-shot), despite using just ~ 3 B parameters and 2-shot in-context examples.

Table from: <https://arxiv.org/pdf/2211.12561>

Multimodal RAG: Results from Yasunaga et al (2022)

Retrieval-Augmented Multimodal Language Modeling

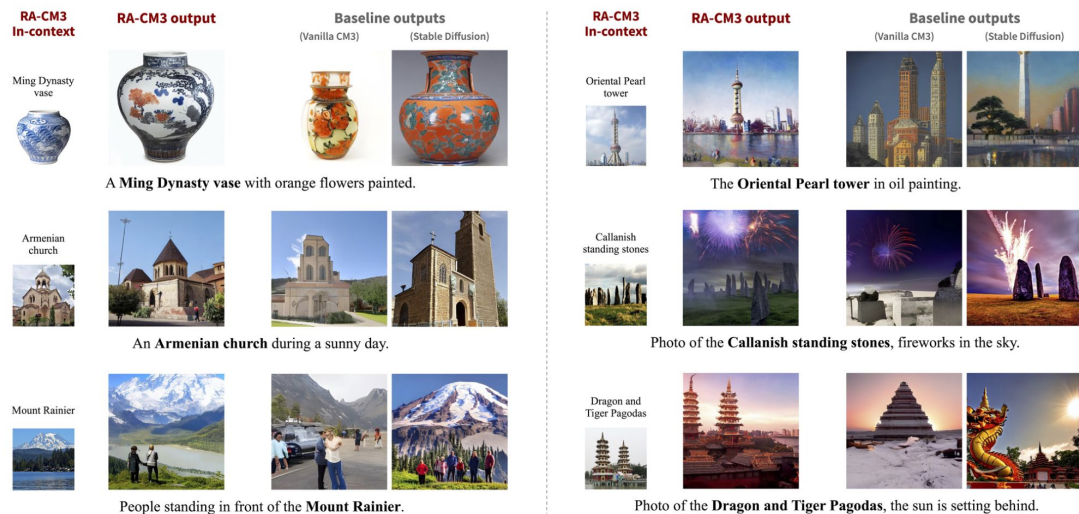


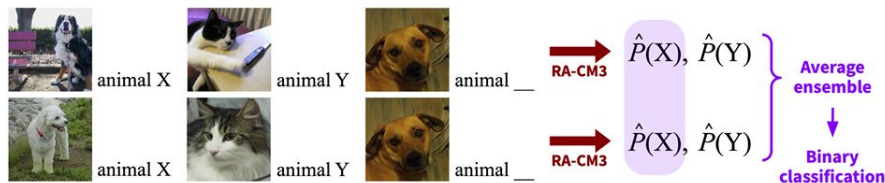
Figure 3. Text-to-image generation involving world knowledge. Our retrieval-augmented model (RA-CM3) can generate correct images from entity-rich captions thanks to the access to retrieved images in the context. For example, RA-CM3’s outputs faithfully capture the visual characteristics of various entities (e.g., the shape and painting of Ming Dynasty vase, the amount of Callanish standing stones). On the other hand, baseline models without retrieval capabilities (vanilla CM3, Stable Diffusion) tend to struggle, especially when the caption involves rare entities (e.g., “Ming Dynasty vase”, “Oriental Pearl tower”, “Dragon and Tiger Pagodas”).

Multimodal RAG: Results from Yasunaga et al.(2022)

One-shot image classification



Few-shot image classification via ensemble (e.g. 2-shot)



Model	<i>k</i> -shot Accuracy			
	<i>k</i> = 1	2	4	8
Baseline CM3	0.53	0.50	0.56	0.56
RA-CM3 (Ours)	0.78	0.79	0.86	0.9

Choice of RAG

	Complexity	Accuracy	Latency	Use Case
Naive	Low	Baseline	Lowest	Simple Queries
Advanced	Medium	Significantly Higher	Low - Medium	Enterprise Search
Modular	High	Max Precision	Moderate	Domain Flexible Systems
Multimodal	Very High	Context Rich	Highest	Non-text Data



Conclusion: RAG

Knowledge Retrieval

Fetches relevant data from external knowledge bases

Factual Precision

Reduces Hallucinations with reliable external sources

Relevancy Scores

Ranks relevancy of each piece of external knowledge

Cost Efficient

Lower cost to implement than retraining

Multimodal Support

Retrieve images, videos, audio, and text

References

- Guu et al. 2020. "REALM: Retrieval-Augmented Language Model Pre-Training".
- Borgeaud et al. 2021. "Improving language models by retrieving from trillions of tokens".
- Khandelwal et al. 2020. "Generalization through Memorization: Nearest Neighbor Language Models"
- Ram et. al 2023 "In-Context Retrieval-Augmented Language Models"
- Shi et al. 2023 "REPLUG: Retrieval-Augmented Black-Box Language Models"
- Zhou et al. 2022 "DocPrompting: Generating Code by Retrieving the Docs"
- Mallen et al. 2023: "When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories"
- Izzacard et al. 2023 : Atlas: Few-shot Learning with Retrieval Augmented Language Models
- Retrieval-Augmented Generation for Large Language Models: A Survey



Thank You!

